

8. To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char input[100];

    while (1) {
        printf("myshell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "pwd") == 0) {
            char *dir = getenv("PWD");

            if (dir)
                printf("%s\n", dir);

            continue;
        }

        if (strncmp(input, "echo $", 6) == 0) {
            char *var = input + 6;
            char *value = getenv(var);

            if (value)
                printf("%s\n", value);
            else
                printf("Undefined variable\n");

            continue;
        }

        printf("Unknown command: %s\n", input);
    }

    return 0;
}
```

[illegible]

9. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <limits.h>

int main() {
    char path[PATH_MAX];
    char current[PATH_MAX];
    char previous[PATH_MAX] = "";

    while (1) {
        getcwd(current, sizeof(current));
        printf("\nCurrent Directory: %s\n", current);

        printf("Enter directory (type exit to quit): ");
        fgets(path, sizeof(path), stdin);
        path[strcspn(path, "\n")] = '\0';

        if (strcmp(path, "exit") == 0)
            break;

        if (strcmp(path, "--") == 0) {
            if (strlen(previous) == 0) {
                printf("No previous directory.\n");
                continue;
            }
            strcpy(path, previous);
        }

        strcpy(previous, current);

        if (chdir(path) == 0) {
            getcwd(current, sizeof(current));
            printf("Directory Changed Successfully.\n");
            printf("New Directory: %s\n", current);
        } else {
            perror("Error");
        }
    }

    return 0;
}
```

```
sharanaya@DESKTOP-TPK9V3Q:~/SKILL$ kill9.c
sharanaya@DESKTOP-TPK9V3Q:~/SKILL$ ./kill9.c -o skill9
sharanaya@DESKTOP-TPK9V3Q:~/SKILL$
```

```
Current Directory: /home/hp/OSSP
Enter directory (type exit to quit): /home/hp
Error: No such file or directory
```

```
Current Directory: /home/hp/OSSP
Enter directory (type exit to quit): exit
saipr@Pranathi:~/SKILL$
```

10. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <limits.h>
#include <sys/wait.h>

int validName(char *name)
{
    if (!(isalpha(name[0]) || name[0] == '_'))
        return 0;

    for (int i = 1; name[i]; i++)
    {
        if (!(isalnum(name[i]) || name[i] == '_'))
            return 0;
    }
    return 1;
}

int main()
{
    char input[200];
    char cwd[PATH_MAX];
    char lastPath[PATH_MAX];

    while (1)
    {
        if (getcwd(cwd, sizeof(cwd)) != NULL)
        {
            strcpy(lastPath, cwd);
            printf("\nCurrent Directory: %s\n", cwd);
        }
        else
        {
            perror("getcwd");
            break;
        }

        printf("\nCommands:\n");
        printf("  pwd\n");
        printf("  export NAME=value\n");
        printf("  child\n");
        printf("  exit\n");
    }
}

```

```

        printf("Exiting program...\n");
        break;
    }

    /* Child Process */
    else if (strcmp(input, "child") == 0)
    {
        pid_t pid = fork();

        if (pid == 0)
        {
            execl("/usr/bin/env", "env", NULL);
            perror("exec");
            exit(1);
        }

        wait(NULL);
    }

    /* Export Command */
    else if (strcmp(input, "export ", 7) == 0)
    {
        char *expr = input + 7;
        char *eq = strchr(expr, '=');

        if (eq == NULL)
        {
            printf("Invalid syntax. Use: export NAME=value\n");
            continue;
        }

        *eq = '\0';
        char *name = expr;
        char *value = eq + 1;

        if (!validName(name))
        {
            printf("Invalid variable name.\n");
            continue;
        }

        setenv(name, value, 1);
        printf("Exported: %s=%s\n", name, getenv(name));
    }
}

```

```

sharanaya@DESKTOP-TPK9V3Q:~/Skill$ kill10.c
sharanaya@DESKTOP-TPK9V3Q:~/Skill$ kill10.c -o skill10
sharanaya@DESKTOP-TPK9V3Q:~/Skill$

```

Current Directory: /home/hp/OSSP

Commands:

```

    pwd
    export NAME=value
    child
    exit

```

Enter command: pwd
Path: /home/hp/OSSP

Current Directory: /home/hp/OSSP

Commands:

```

    pwd
    export NAME=value
    child
    exit

```

Enter command: export
Invalid syntax. Use: export NAME=value

Current Directory: /home/hp/OSSP

Commands:

```

    pwd
    export NAME=value
    child
    exit

```

Enter command: █